

PROLAB_2 1.PROJE RAPORU

Hayrunisa KORKULU ve Farahnozkhon Dilorovna MASUMOVA

1. Ulaşım Rota Planlama Sistemi

1.0. Özet

Bu projede, İzmit şehri için JavaFX tabanlı geliştirilen bir rota planlama sisteminin tasarımı ve uygulanması sunulmaktadır. Sistem, kullanıcıdan alınan başlangıç konumu (enlem-boylam), hedef durak, yolcu tipi (genel, öğrenci, öğretmen, yaşlı), ödeme yöntemi (nakit, kredi kartı, kentkart), rota filtresi (en kısa, en hızlı, minimum aktarma) ve taşıt türü (otobüs, tramvay, taksi) gibi bilgileri alarak en uygun rotayı hesaplamaktadır.

Uygulama, JSON formatındaki gerçek verisetini kullanarak duraklar arasındaki mesafeleri, süreleri, ücretleri ve transfer bilgilerini işleyebilmekte; kullanıcıya hem Google Maps WebView ile harita üzerinde, hem de mxGraph ile grafiksel görselleştirme sağlamaktadır.

Proje, nesne yönelimli programlama prensiplerine tam uyumlu şekilde geliştirilmiştir. SOLID ilkeleriyle yapılandırılan kod mimarisi, Strategy ve Factory tasarım desenleri sayesinde yeni araç türlerinin veya ödeme yöntemlerinin sisteme kolaylıkla entegre edilmesini sağlamaktadır. Hesaplamalarda Haversine algoritması kullanılarak en yakın durak tespiti yapılmış, toplu taşıma için transfer ve aktarma maliyetleri dahil edilmiştir.

Bu rota planlama sistemi, kullanıcıya çok yönlü filtreleme imkânı sunarak şehir içi ulaşım deneyimini daha verimli hale

getirmekte; modüler yapısı sayesinde farklı senaryolara kolayca uyarlanabilmektedir.

2.0 GİRİŞ

Günümüz şehirlerinde artan nüfus, karmaşılaşan ulaşım ağları ve zamanın verimli kullanımı ihtiyacı, rota planlama sistemlerinin önemini daha da artırmaktadır. Özellikle farklı yolcu profilleri ve çeşitli taşıma araçlarının bir arada bulunduğu ortamlarda, kullanıcıya en uygun ve ekonomik güzergâhı sunmak şehir içi ulaşımın kalitesini doğrudan etkilemektedir.

Bu bağlamda geliştirilen İzmit Rota Planlama Sistemi, kullanıcıdan alınan konum ve tercih bilgilerine dayanarak en ideal güzergâhı belirlemeyi amaçlayan JavaFX tabanlı bir masaüstü uygulamasıdır. Sistem, kullanıcıya Google Haritalar üzerinde yolculuk başlangıcı ve bitişini gösterirken, aynı zamanda grafiksel bir ağ üzerinden (mxGraph) geçilen durakları ve kullanılan taşıma türlerini de görsel olarak sunmaktadır.

Projenin temel motivasyonu; kullanıcı odaklı, genişletilebilir ve sürdürülebilir bir ulaşım çözümü sunmaktır. Ayrıca yazılım mühendisliği prensiplerine uygun olarak geliştirilen bu sistem, benzer projeler için yeniden kullanılabilir bir altyapı sağlamaktadır. Uygulama, üniversite öğrencilerinden yaşlı bireylere kadar her

kullanıcı profiline hitap edecek şekilde tasarlanmıştır.

3.0 YÖNTEM

3.1 Sistem Mimarisi

Sistem, JavaFX arayüzü ile kullanıcıdan veri alır ve bu verileri kullanarak rota hesaplama işlemini başlatır. Uygulama aşağıdaki temel modüllerden oluşur:

- **Veri Girişi Modülü:** Kullanıcıdan enlem-boylam, yolcu tipi, ödeme yöntemi, hedef durak ve taşıma türü gibi bilgiler alınır.
- **Rota Hesaplama Modülü:** Genişletilebilir yapıdaki UlasimAgi sınıfı içerisinde rota hesaplamaları yapılır.
- **Görselleştirme Modülü:** Google Maps (WebView) ve mxGraph (SwingNode) kullanılarak rota çizimi yapılır.
- **Veri Kaynağı:** Tüm durak ve bağlantı bilgileri, veriseti.json adlı JSON dosyasından yüklenir.

3.2 Kullanılan Algoritma ve Hesaplamalar

- **Mesafe Hesaplama:** Kullanıcının enlem ve boylam bilgisine göre en yakın durak, Haversine algoritması ile belirlenir.
- **Rota Seçimi:** BFS (Breadth-First Search) algoritması ile duraklar arası en uygun yol aranır. Her yol için süre, aktarma sayısı ve ücret hesaplanır.
- **Ücret Hesaplama:** Yolcu tipine göre (Öğrenci, Öğretmen, Yaşlı vb.) taşıtlardaki indirim oranları

uygulanır. Taksi için sabit açılış ücreti + km başına ücret hesaplanır.

- **Transfer İşlemleri:** JSON verisindeki transfer alanı kontrol edilerek, iki farklı taşıt türü arasında geçiş yapılır. Süre ve ücret bilgisi buna göre eklenir.

3.3 Tasarım Desenleri

- **Strategy Deseni:** Mesafe hesaplama işlemi için MesafeHesaplamaStrategy arayüzü ve HaversineMesafe sınıfı kullanılmıştır.
- **Factory Deseni:** Araç türlerine özel ücretlendirme işlemleri için AracFactory sınıfı tanımlanmıştır. Otobüs, tramvay, taksi gibi nesneler burada oluşturulur.

3.4 Grafikselsel Rota Gösterimi ve Kütüphaneler

- tumDuraklariVeRotaCiz fonksiyonu ile mxGraph kullanılarak duraklar arasında bağlantılar renkli oklarla çizilir.
- Araç türüne göre kenar rengi değişir: otobüs (yeşil), tramvay (mavi), taksi (turuncu), transfer (kırmızı).
- Taksi başlangıç/son noktaları ikon ve etiketle vurgulanır.
- mxGraph kütüphanesi ile JavaFX'in SwingNode bileşeni entegre edilerek hem stil hem interaktivite sağlanmıştır.
- Harita görünümü için JavaFX WebView ve Google Maps URL'leri ile canlı harita yönlendirme desteği sunulmuştur.

4.0. DENEYSEL SONUÇLAR

Uygulama, İzmit şehrine ait gerçek durak verileri ile test edilmiştir. Kullanıcının başlangıç konumu olarak girdiği koordinatlardan en yakın durağa olan mesafe Haversine algoritmasıyla hesaplanmış ve bu mesafe 3 kilometreden fazla ise taksi önerisi otomatik olarak rota başlangıcına eklenmiştir. Aynı şekilde, hedef durağa ulaşımın ardından gerçek varış noktasına olan mesafe de kontrol edilerek gerekli durumlarda ikinci bir taksi segmenti sisteme dahil edilmiştir.

Sistemin başarılı çalıştığı bazı örnek senaryolar aşağıdaki gibidir:

- **Senaryo 1:** Başlangıç noktası olarak Otogar (40.78259, 29.94628) koordinatları girilmiş ve hedef olarak "bus_41burda" seçilmiştir. Yolcu tipi "Öğrenci", taşıt tercihi "Otobüs + Tramvay" ve rota filtresi "Minimum Aktarma" seçildiğinde, sistem rota üzerinde aktarmaları da hesaplayarak öğrenci indirimi uygulanmış bir ücret sunmuştur. Sonuçlar: 18 dakika, 6.50 TL.
- **Senaryo 2:** Başlangıç noktası Sekapark (40.76520, 29.96190), hedef ise "tram_halkevi" olarak girildiğinde, sistem kullanıcının önce tramvaya binmesini ve Sekapark'tan Halkevi'ne aktarma yapılmasını önermiştir. Tramvay hattının tamamı başarıyla görselleştirilmiş, toplam süre 7 dakika ve ücret 1.5 TL olarak belirlenmiştir.

- **Senaryo 3:** Kullanıcı Umuttepe'ye (bus_umuttepe) gitmek istediğinde sistem, tramvay ve otobüs aktarmalarını içeren karma bir rota sunmuş; yolculuk toplamda 22 dakika ve 9.00 TL olarak hesaplanmıştır. Transfer noktaları grafiksel olarak kırmızı kenarlarla, taşıtlar ise uygun renklerle gösterilmiştir.

Uygulama, JSON veri setine sadık kalarak her senaryoda doğru süre, mesafe ve ücret hesaplaması sağlamış ve grafiksel arayüzde kullanıcıya anlaşılır bir çıktıyla sunmuştur. Kullanıcıya her adımda geçilen duraklar, taşıma türleri ve ödeme tutarları detaylı şekilde gösterilmiş; hem harita hem de grafik sekmelerinde güncel ve tutarlı bilgi aktarımı gerçekleştirilmiştir.

5.0. PSEUDOCODE

1.0. Gerekli Sınıfları Tanımla

1.1. Durak Sınıfı (id, name, type, lat, lon, nextStops, transfer)

Metotlar: `mesafeHesapla()`, `hasTransfer()`, `getTransferUcret()`, `getTransferSure()`

1.2. Yolcu Sınıfı (soyut)

Alt sınıflar: `GenelYolcu`, `Ogrenci`, `Ogretmen`, `Yasli`
Metotlar: `indirimliUcret()`

1.3. Odeme Sınıfı (soyut)

Alt sınıflar: `Nakit`, `KrediKarti`, `KentKart`
Metotlar: `odemeYap()`

1.4. Taksi Sınıfı

Değişkenler: `acilisUcreti`, `kmUcreti`
Metotlar:

ucretHesapla(mesafe, yolcuTipi)

1.5. AracFactory Sınıfı

Metotlar: getArac(String type) → Gerekli taşıt nesnesini döndürür

1.6. MesafeHesaplamaStrategy Arayüzü

Uygulayan:

HaversineMesafe

Metot: mesafeHesapla(x1, y1, x2, y2)

2.0. Ana Veri Yapısı ve Rota İşleyici

2.1. UlasimAgi Sınıfı

Veriler: Map<String, Durak> duraklar, graf

Metotlar: rotaHesapla(...), enYakinDurak(), tumDuraklariVeRotaCiz(...)

2.2. RotaSonucu Sınıfı

Veriler: rota, toplamUcret, toplamSure, aracTurleri

3.0. Arayüz Yapısı (GUI)

3.1. GirisEkrani.java

Kullanıcıdan enlem, boylam, hedef durak, yolcu tipi, ödeme yöntemi, taşıt filtresi gibi girdileri alır

Harita sekmesi: Google Maps (WebView)

Graf sekmesi: mxGraph (SwingNode ile)

4.0. Algoritmanın Temel Adımları

4.1. Kullanıcı konumuna en yakın durağı enYakinDurak() ile hesapla

4.2. Eğer başlangıç mesafesi > 3 km → taksi başlangıcı ekle

4.3. BFS başlat: Kuyruğa başlangıç durağını ekle

4.4. Kuyruk boş değilken:

- Bir yol çıkar
- Son durağa ulaşıldıysa

ücret/süre/aktarma hesapla

- JSON üzerinden veri çek,

AracFactory ile araç türüne göre hesapla

- Transfer varsa ayrı ele al
- Eğer filtreye göre bu yol daha

iyiyse güncelle

4.5. Aksi halde:

- Komşu durakları kontrol et
- uygunArac(...) fonksiyonu ile taşıt filtresi kontrol et

- Yol büyütülerek kuyruğa eklenir

4.6. Eğer hedef durağa olan mesafe > 3 km → taksi hedefi ekle

4.7. Rota sonucu RotaSonucu nesnesine dökülür

5.0. Rota Çizimi ve Harita

5.1. tumDuraklariVeRotaCiz()

fonksiyonu:

- mxGraph ile tüm durakları sırayla çizer
- Araç türüne göre renk: Otobüs (mavi), Tramvay (yeşil), Taksi (turuncu), Transfer (kırmızı)
- Taksi için özel etiketler ve ikonlar eklenir

5.2. WebView üzerinden Google Maps rotası gösterilir

URL formatı:

<https://www.google.com/maps/dir/lat1,lon1/lat2,lon2>

6.0. Ek Özellikler ve Genişletilebilirlik

6.1. Kullanıcıya araç seçimi, ödeme tipi, yolcu indirimi gibi özel filtreleme

6.2. SOLID prensiplerine göre:

- Yeni taşıt türleri (örneğin elektrikli scooter) sadece AracFactory ve yeni sınıf ile eklenebilir
- Var olan algoritmalarda değişiklik gerekmez

6.3. 65 yaş üstü için seyahat limiti:

- Yolcu sınıflarına seyahatSayisi eklenerek kontrol edilebilir

- Ücret hesaplaması indirimliUcret() içinde yönetilebilir

6.0 SONUÇ

Bu çalışmada, şehir içi ulaşımı kolaylaştırmak ve kullanıcıya en uygun güzergahı sunmak amacıyla geliştirilen İzmit Rota Planlama Sistemi ayrıntılı olarak sunulmuştur. JavaFX tabanlı bu uygulama, farklı yolcu tiplerine göre dinamik ücret hesaplamaları, taşıma türü filtrelemeleri, transfer işlemleri ve harita üzerinde canlı yönlendirme özellikleriyle donatılmıştır. Gerçek bir veri seti ile test edilen sistem; doğruluk, esneklik ve kullanıcı dostu arayüz açısından başarılı sonuçlar vermiştir.

Sistem mimarisi, SOLID prensiplerine uygun olarak modüler ve genişletilebilir biçimde tasarlanmıştır. Strategy ve Factory gibi tasarım desenleri sayesinde yeni taşıt türleri veya ödeme seçenekleri eklenebilir hâle getirilmiştir. Özellikle yolculuk hesaplamalarında transfer noktaları, araç türleri, mesafe/süre/ücret ilişkileri ve kullanıcı indirimleri gibi faktörler, algoritmanın hassas çalışmasını sağlamıştır.

Grafiksel görselleştirme, hem yön bulmayı kolaylaştırmış hem de kullanıcıya yolculuk sürecini daha anlaşılır kılmıştır. Geliştirilen sistem sadece İzmit şehrine değil, benzer yapıda veri setiyle farklı şehir ulaşım sistemlerine de kolayca adapte edilebilir yapıdadır.

Bu proje ile birlikte, kullanıcı merkezli, akıllı ve genişletilebilir bir rota planlama çözümü başarıyla gerçekleştirilmiş; gerçek dünya verisiyle uyumlu çalışan bir toplu taşıma modellemesi sunulmuştur. İlerleyen çalışmalarla birlikte, otobüs saatleri, trafik

yoğunluğu gibi dinamik veriler de dahil edilerek sistem daha da geliştirilebilir.

7.0 SUNUM SORULARI VE CEVAPLARI

Soru 1: Yeni bir ulaşım yöntemi (örneğin, elektrikli scooter) eklenmek istendiğinde sistemde ne tür değişiklik yapılmalıdır?

Cevap: Yeni ulaşım türleri sisteme kolayca entegre edilebilir. Bunun için Arac soyut sınıfından Scooter gibi yeni bir sınıf türetilmelidir. Bu sınıfta ucretHesapla() gibi metodlar yolcu tipine özel olarak yeniden tanımlanır. AracFactory'ye sadece yeni bir koşul eklenmesi yeterlidir. Mevcut sınıfların değiştirilmesine gerek kalmaz.

Soru 2: Otonom taksi gibi gelişmiş araçlar için ne tür değişiklikler gerekir?

Cevap: Taksi sınıfı genişletilerek OtonomTaksi gibi bir sınıf oluşturulabilir. Bu sınıf içinde özel kurallar (örneğin gece indirim, otomatik rotalama gibi) uygulanabilir. AracFactory bu sınıfı üretmeye uygun hale getirilmelidir.

Soru 3: Daha önce yazılmış hangi fonksiyonlar etkilenir?

Cevap: Sadece AracFactory sınıfına yeni bir case eklenmesi gerekir. UlasimAgi sınıfı içerisindeki rotaHesapla() metodunda araç nesnesi AracFactory üzerinden çağrıldığı için başka bir yerde değişikliğe gerek kalmaz. indirimliUcret() metodunun da kapsayıcı olması gereklidir.

Soru 4: Open/Closed prensibine göre yeni araçlar mevcut kodu bozmadan nasıl eklenir?

Cevap: Arac soyut sınıfı genişletilerek yeni sınıflar (örneğin Scooter) oluşturulabilir. AracFactory sınıfında yeni sınıfı üretmek için yeni bir case eklenir.

Diğer tüm kodlar değişmeden kalır. Böylece kod açık (extend) ama kapalı (modifiye) prensibine uygun olur.

Soru 5: 65 yaş üzeri kullanıcılar için ücretsiz seyahat hakkı 20 seyahat ile sınırlandırılırsa ne olur?

Cevap: Yasli sınıfına seyahatSayisi adında bir sayaç eklenir. indirimliUcret() metoduna bu sayaç kontrolü entegre edilir. Eğer sayaç 20'den büyükse tam ücret uygulanır. Sayaç her rota sonrası bir artırılır.

Soru 6: Bu sınırlama hangi sınıf ve metodlarda uygulanmalıdır?

Cevap: Sınırlama doğrudan Yasli sınıfında uygulanmalı; indirimliUcret() içinde sayaç kontrolü yapılmalıdır. Sayaç değeri her başarılı rotaHesapla() işlemi sonunda güncellenir. Böylece diğer yolcu türleri etkilenmeden sadece Yasli sınıfı içinde bu özellik tanımlanmış olur.

8.0 YAZAR KATKILARI

Hayrunisa Korkulu ; projeye başlamadan önce gerekli araştırmaları yaparak hangi kütüphanelerin kullanılacağını belirlemiştir. JavaFX arayüz tasarımı, giriş ekranı ve kullanıcıdan veri alma işlemlerini gerçekleştirmiştir. UlasimAgi sınıfının oluşturmuştur. JSON verisinin parse edilerek grafik yapısına dönüştürülmesini sağlamıştır. SOLID prensiplerinin uygulanması: Açık-Kapalı, Tek Sorumluluk gibi prensipler ile genişletilebilirlik sağlamıştır. Google Maps WebView ve mxGraph kütüphanelerinin GUI'ye entegre etmiştir. Giriş ekranında dinamik olarak hedef durak listesinin ComboBox ile gösterilmesi ve kullanıcının seçim yapabilmesini sağlamıştır. Arayüzdeki tüm ComboBox, WebView,

harita yükleme ve grafik çizim işlevlerinin entegre edilmesini sağlamıştır. Grafik ekranının genişliği, kenar çizim stilleri, simge kullanımı gibi görsellik detayları ayarlayarak görüntünün daha anlaşılır ve dikkat çekici hale gelmesini sağlamıştır.

Farahnozkhon Dilarovna Masumova ;

projeye başlamadan önce gerekli araştırmaları yaparak hangi kütüphanelerin kullanılacağını belirlemiştir. Nesne yönelimli programlama ilkelerine (OOP) uygun şekilde Durak, Yolcu, Arac, Taksi, Otobus, Tramvay, Odeme gibi sınıflarını doğru hiyerarşide tanımlamıştır. Rota hesaplama algoritmasını geliştirmiştir (BFS + maliyet-süre hesaplamaları). Mesafe hesaplamaları için Strategy Pattern ve araç ücretlendirmesi için Factory Pattern'in uygulanmasını sağlamıştır. Taksi entegrasyonu ve başlangıç/hedef noktaları için mesafe kontrolünü yapmıştır. Tüm kullanıcı tipleri için ücret indirimleri ve özel durumların işlenmesini sağlamıştır. Farklı yolcu türlerine göre ücret indirimlerinin uygulanması ve ödeme yöntemleri sınıflarını işlevsel hale getirmiştir. Projenin test aşamasında çıkan hataları büyük bir özenle gidermiş ve projenin düzgün çalışmasına katkı sağlamıştır.

9.0 KAYNAKÇA

[1] Oracle, "JavaFX Documentation,"

[Çevrimiçi]. Erişim adresi:

<https://openjfx.io>

[2] Google Developers, "Google Maps

JavaScript API," [Çevrimiçi]. Erişim adresi:

<https://developers.google.com/maps/documentation>

[3] JGraph Ltd., "mxGraph Java Swing

Library," [Çevrimiçi]. Erişim adresi:

<https://github.com/jgraph/mxgraph>

[4] OpenAI, "ChatGPT – AI language model," [Çevrimiçi]. Erişim adresi:

<https://chat.openai.com>

[5] Cursor AI, "Cursor Code Editor with AI Assistance," [Çevrimiçi]. Erişim adresi:

<https://www.cursor.so>

[6] Kocaeli Üniversitesi, Programlama Laboratuvarı Ders Notları.

[7] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.